PROGRAMAÇÃO ORIENTADA À OBJETOS

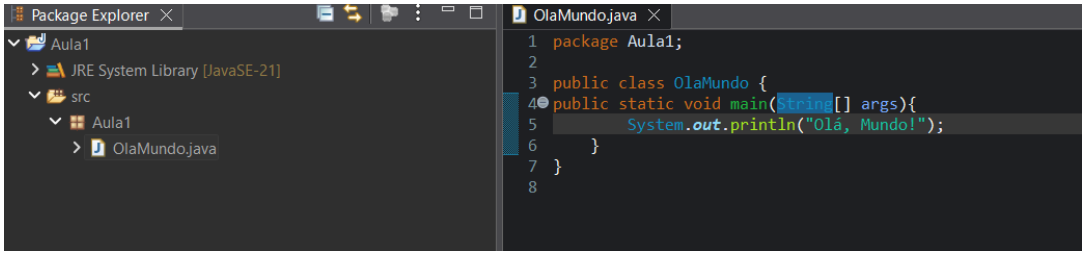
- 2 provas e 1 trabalho
- Ele vai deixar os slides, exercícios e livros no teams

TUTORIAL DE IDE

ECLIPSE

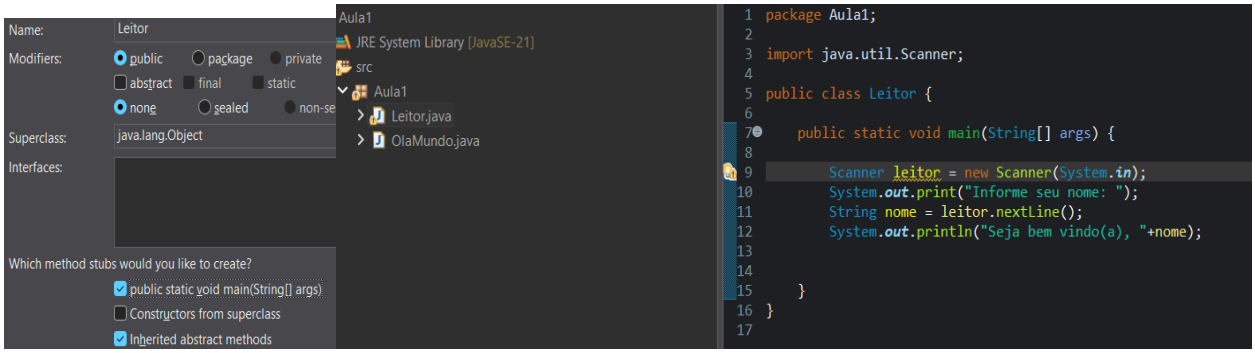
Diferente de uma RAD, onde o objetivo é desenvolver o mais rápido possível através do arrastar-e-soltar do mouse, onde montanhas de código são gerados em background, uma IDE te auxilia no desenvolvimento, evitando se intrometer e fazer muita mágica

- Baixar Eclipse (também funciona em outras IDEs)
- New Java Project → New package “Aula1” → New class “OlaMundo”
- Tirar o checkbox “Create module- …”



LEITURA

- Scanner leitor = new Scanner(System.in);
- Precisa de um input



- Next: captura; NextLine: captura e quebra linha
- E/S COM JOptionPane: Janelas

INTRODUÇÃO

PROGRAMAÇÃO ORIENTADA À OBJETOS

- A POO é um dos fundamentos mais importantes para quem deseja escrever código Java mais organizado, reutilizável e fácil de manter.
- Pilares: encapsulamento, herança, polimorfismo e abstração — ajudam a construir aplicações mais limpas, seguras e escaláveis, além de transformar a forma como pensamos o desenvolvimento de software.

POR QUE USAR?

- Reutilização → evitamos repetir código, aproveitando classes e métodos já criados
- Facilidade de manutenção → mudanças ficam isoladas, sem quebrar o sistema todo
- Organização → o código reflete melhor a realidade, facilitando o entendimento
- Escalabilidade → projetos crescem de forma sustentável
- Segurança → protegemos dados sensíveis usando encapsulamento

MODIFICADORES DE ACESSO

- Public → acesso liberado de qualquer lugar
- Protected → acesso dentro da mesma classe, mesmo pacote e também por subclasses (mesmo que estejam em pacotes diferentes)
- Private → acesso restrito somente dentro da própria classe

ENCAPSULAMENTO → esconder os detalhes internos de um objeto e expor apenas o que é necessário, controlando o acesso por meio de modificadores de acesso

```java
class Personagem {
    // Detalhes internos (encapsulados)
    private String nome;
    private int poderDeLuta;

    // Construtor
    public Personagem(String nome, int poderDeLuta) {
        this.nome = nome;
        this.poderDeLuta = poderDeLuta;
    }

    // Expõe o que é necessário
    public String getNome() {
        return nome;
    }

    // Expõe de forma controlada
    public int getPoderDeLuta() {
        return poderDeLuta;
    }

    // Controla como o poder pode ser alterado
    public void treinar(int aumento) {
        if (aumento > 0) {
            this.poderDeLuta += aumento;
        }
    }
}
```

```java
// Superclasse
class LutadorZ {
    protected String nome;
    protected int poderLuta;

    public void apresentar() {
        System.out.println("Eu sou " + nome + " e meu poder de luta é " + poderLuta);
    }
}

// Subclasse: Sayajin
class Sayajin extends LutadorZ {
    private boolean transformadoSuperSayajin;

    @Override
    public void apresentar() {
        System.out.println("Sayajin lendário " + nome + " com poder de " + poderLuta);
    }

    public void transformar() {
        if (!transformadoSuperSayajin) {
            transformadoSuperSayajin = true;
            poderLuta *= 50;
            System.out.println(nome + " se transformou em Super Sayajin!");
        }
    }
}
```

HERANÇA → quando uma classe filha (subclasse) herda atributos e métodos da classe pai (superclasse), para reaproveitar código e poder especializar comportamentos

---

BASE JAVA

COMENTÁRIOS

- Você pode usar o // para comentar até o final da linha, ou então usar o /* */ para comentar o que estiver entre eles.

```java
/* comentário daqui,
ate aqui */

// uma linha de comentário sobre a idade
int idade;
```

DECLARANDO E USANDO VARIÁVEIS

- Conceito: São as características (propriedades) ou dados que a classe armazena.
- Dentro de um bloco, podemos declarar variáveis e usá-las. Em Java, toda variável tem um tipo que não pode ser mudado, uma vez que declarado: tipoDaVariavel nomeDaVariavel;
- Por exemplo, é possível ter uma idade que guarda um número inteiro: int idade;
- Com isso, você declara a variável idade, que passa a existir a partir daquela linha. Ela é do tipo int, que guarda um número inteiro. A partir daí, você pode usá-la, primeiramente atribuindo valores.

- Além de atribuir, você pode utilizar esse valor. O código a seguir declara novamente a variável idade com valor 15 e imprime seu valor na saída padrão através da chamada a System.out.println.

```java
// declara a idade
int idade;
idade = 15;

// imprime a idade
System.out.println(idade);
```

```java
// calcula a idade no ano seguinte
int idadeNoAnoQueVem;
idadeNoAnoQueVem = idade + 1;
```

- Por fim, podemos utilizar o valor de uma variável para algum outro propósito, como alterar ou definir uma segunda variável.

- O código a seguir cria uma variável chamada idadeNoAnoQueVem com valor de idade mais um.

- Convenção:
  - Ele deve começar com uma letra minúscula, como id, nome.
  - Não deve começar com caracteres especiais como & (e comercial), $ (dólar), _ (sublinhado).
  - Se o nome contiver várias palavras, inicie-o com a letra minúscula seguida por uma letra maiúscula como firstName, lastName.
  - Evite usar variáveis de um caractere, como x, y, z.

OPERADORES

- No mesmo momento que você declara uma variável, também é possível inicializá-la por praticidade: int idade = 15;
- Você pode usar os operadores +, -, / e * para operar com números, sendo eles responsáveis pela adição, subtração, divisão e multiplicação, respectivamente.
- Além desses, há o operador % (módulo) ou o resto de uma divisão inteira.
- Exemplos:

```
int quatro = 2 + 2;
int tres = 5 - 2;

int oito = 4 * 2;
int dezesseis = 64 / 4;

int um = 5 % 2; // 5 dividido por 2 dá 2 e tem resto 1;
                // o operador % pega o resto da divisão inteira
```

COMO RODAR ESSES CÓDIGOS

- Você deve colocar esses trechos de código dentro do bloco main. Use bastante System.out.println para ver os resultados.
- Exemplo:

```
class TestaIdade {

    public static void main(String[] args) {
        // imprime a idade
        int idade = 20;
        System.out.println(idade);

        // gera uma idade no ano seguinte
        int idadeNoAnoQueVem;
        idadeNoAnoQueVem = idade + 1;

        // imprime a idade
        System.out.println(idadeNoAnoQueVem);
    }
}
```

TIPOS DOUBLE E BOOLEAN

- O tipo double armazena números com ponto flutuante: double pi = 3.14;
- O tipo boolean armazena apenas true (verdadeiro) ou false (falso).
- Exemplo de expressão booleana:

```
int idade = 30;
boolean menorDeIdade = idade < 18;
```

```
char letra = 'a';
```

TIPO CHAR

- O tipo char guarda um, e apenas um, caracterizado entre aspas simples.

- Não pode guardar um valor vazio ".
- Strings (cadeias de caracteres) não são tipos primitivos e serão vistas adiante

TIPOS PRIMITIVOS E VALORES

- Em tipos primitivos, o valor é o real conteúdo da variável.
- Ao usar o operador de atribuição =, o valor é copiado.

```
int i = 5; // i recebe uma cópia do valor 5
int j = i; // j recebe uma cópia do valor de i
i = i + 1; // i vira 6, j continua 5
```

- Aqui, i fica com o valor de 6. Mas e j ? Na segunda linha, j está valendo 5. Quando i passa a valer 6, será que j também muda de valor? Não, pois o valor de um tipo primitivo sempre é copiado.
- Se fizermos j = i, as variáveis deixam de ter relação; o que acontece com uma não reflete na outra.

CONVENÇÕES JAVA

- https://www.oracle.com/java/technologies/java se/codeconventions-contents.html
- Ao usar as convenções de nomenclatura padrão do Java, você facilita a leitura do código para você e outros programadores. A legibilidade do programa Java é muito importante. Isso indica que menos tempo é gasto para descobrir o que o código faz.
- A seguir, estão as principais regras que devem ser seguidas por todos os identificadores:
    - O nome não deve conter espaços em branco.
    - O nome não deve começar com caracteres especiais como & (e comercial), $ (dólar), _ (sublinhado)

CLASSE

- Conceito: em Java (e em muitas outras linguagens de POO), é um modelo ou blueprint a partir do qual objetos são criados. Ela define um conjunto de propriedades (atributos) e comportamentos (métodos) que os objetos desta classe terão
- Convenção:
    - Deve começar com a letra maiúscula.
    - Deve ser um substantivo como Cor, Botão, Sistema, Tópico, etc.
    - Use palavras apropriadas, em vez de siglas.

```
public class Employee {
    //fragmento de código
}
```

MÉTODO

- Conceito: São as funções ou operações que a classe pode realizar. Eles geralmente operam nos atributos da classe ou executam ações relacionadas a esses atributos. No caso da classe Car, métodos poderiam ser acelerar(), frear(), etc.
- Convenção:
    - Deve começar com letra minúscula.
    - Deve ser um verbo como main(), print(), println().
    - Se o nome contiver várias palavras, inicie-o com uma letra minúscula seguida por uma letra maiúscula, como actionPerformed()

```
class Employee {
    //método
    void desenhar() {
        //fragmento de código
    }
}
```

```
class Employee {
    //variável
    int id;
    //fragmento de código
}
```

PACOTE

- Deve ser uma letra minúscula, como java, lang.
- Se o nome contiver várias palavras, ele deverá ser separado por pontos (.) Como java.util, java.lang.

```
package com.javatpoint; //pacote
class Employee {
    //fragmento de código
}
```

CONSTANTE

- Deve estar em letras maiúsculas, como VERMELHO, AMARELO.
- Se o nome contiver várias palavras, ele deverá ser separado por um sublinhado (_), como MAX_PRIORITY.
- Pode conter dígitos, mas não como a primeira letra.

```
class Employee {
    //constante&
    final static int MIN_AGE = 18 ;
    //fragmento de código
}
```

CAMELCASE EM CONVENÇÕES DE NOMENCLATURA JAVA

- Java segue a sintaxe camelCase para nomear a classe, interface, método e variável.
- Se o nome for combinado com duas palavras, a segunda palavra começará com letras maiúsculas sempre como actionPerformed(), firstName, ActionEvent, ActionListener, etc.

CASTING E PROMOÇÃO

- Alguns valores são incompatíveis se você tentar fazer uma atribuição direta. Enquanto um número real costuma ser representado em uma variável do tipo double , tentar atribuir ele a uma variável int não funciona porque é um código que diz: "i deve valer d", mas não se sabe se ‘d’ realmente é um número inteiro ou não.
- O mesmo ocorre no seguinte trecho
- O mais interessante, é que nem mesmo o seguinte código compila:

```
double d = 3.1415;
int i = d; // não compila
```

```
int i = 3.14;
```

```
double d = 5; // ok, o double pode conter um número inteiro
int i = d; // não compila
```

- Apesar de 5 ser um bom valor para um int , o compilador não tem como saber que valor estará dentro desse double no momento da execução. Esse valor pode ter sido digitado pelo usuário, e ninguém vai garantir que essa conversão ocorra sem perda de valores.
- Já no caso a seguir, é o contrário:

```
int i = 5;
double d2 = i;
```

- O código acima compila sem problemas, já que um double pode guardar um número com ou sem ponto flutuante. Todos os inteiros representados por uma variável do tipo int podem ser guardados em uma variável double, então não existem problemas no código acima.
- Às vezes, precisamos que um número quebrado seja arredondado e armazenado num número inteiro

- Para fazer isso sem que haja o erro de compilação, é preciso ordenar que o número quebrado seja moldado (casted) como um número inteiro. Esse processo recebe o nome de casting
- O casting foi feito para moldar a variável d3 como um int . O valor de i agora é 3.
- O mesmo ocorre entre valores int e long

```
double d3 = 3.14;
int i = (int) d3;
```

```
long x = 10000;
int i = x; // não compila, pois pode estar perdendo informação
```

- E, se quisermos realmente fazer isso, fazemos o casting:

```
long x = 10000;
int i = (int) x;
```

CASTINGS POSSÍVEIS

- Abaixo estão relacionados todos os casts possíveis na linguagem Java, mostrando a conversão de um valor para outro. A indicação Impl. quer dizer que aquele cast é implícito e automático, ou seja, você não precisa indicar o cast explicitamente (lembrando que o tipo boolean não pode ser convertido para nenhum outro tipo).

PARA: / DE:	byte	short	char	int	long	float	double
byte	----	*Impl.*	(char)	*Impl.*	*Impl.*	*Impl.*	*Impl.*
short	(byte)	----	(char)	*Impl.*	*Impl.*	*Impl.*	*Impl.*
char	(byte)	(short)	----	*Impl.*	*Impl.*	*Impl.*	*Impl.*
int	(byte)	(short)	(char)	----	*Impl.*	*Impl.*	*Impl.*
long	(byte)	(short)	(char)	(int)	----	*Impl.*	*Impl.*
float	(byte)	(short)	(char)	(int)	(long)	----	*Impl.*
double	(byte)	(short)	(char)	(int)	(long)	(float)	----

TAMANHO DOS TIPOS

TIPO	*TAMANHO*
boolean	1 bit
byte	1 byte
short	2 bytes
char	2 bytes
int	4 bytes
float	4 bytes
long	8 bytes
double	8 bytes

EXERCÍCIOS

1) Na empresa onde trabalhamos, há tabelas com o quanto foi **gasto em cada mês**. Para fechar o balanço do primeiro trimestre, precisamos somar o gasto total. Sabendo que, em Janeiro, foram gastos 15000 reais, em Fevereiro, 23000 reais e em Março, 17000 reais, faça um programa que calcule e imprima o gasto total no trimestre. Siga esses passos:

- Crie uma classe chamada BalancoTrimestral com um bloco main, como nos exemplos anteriores;
- Dentro do main (o miolo do programa), declare uma variável inteira chamada gastosJaneiro e inicialize-a com 15000;
- Crie também as variáveis gastosFevereiro e gastosMarco , inicializando-as com 23000 e 17000, respectivamente, utilize uma linha para cada declaração;
- Crie uma variável chamada gastosTrimestre e inicialize-a com a soma das outras 3 variáveis: **int gastosTrimestre = gastosJaneiro + gastosFevereiro + gastosMarco**;
- Imprima a variável gastosTrimestre .

2) Adicione código (sem alterar as linhas que já existem) na classe anterior para imprimir **a média mensal de gasto**, criando uma variável mediaMensal junto com uma mensagem. Para isso, concatene a String com o valor, usando "Valor da média mensal = " + mediaMensal.

3) Crie um programa que exiba a **tabuada** de um número informado pelo usuário;

4) Crie um programa que **troque os valores** que estão armazenados nas variáveis nome1 e nome2.

- A variável nome1 armazenara o valor "mestre do universo".
- A variável nome2 receberá um valor informado pelo usuário.
- Altere os valores das variáveis e exiba a soma da variável nome1 com o valor da variável nome2.

5) Crie um programa que **calcule a média de um aluno** para a matéria de matemática. O aluno deve informar seu nome e as quatro notas da matéria;

6) Escreva um programa que, com base em uma **temperatura** em graus celsius, a converta e exiba em Kelvin (K), Réaumur (Re), Rankine (Ra) e Fahrenheit (F), seguindo as fórmulas: F = C * 1.8 + 32; K = C + 273.15; Re = C * 0.8; Ra = C * 1.8 + 32 + 459.67

7) Calcular e apresentar o valor do volume de uma **lata de óleo**, utilizando fórmula: V = 3.14159 * R * R * A, em que as variáveis: V, R e A representam respectivamente o volume, o raio e a altura. Vamos declarar as variáveis V, R e A como double e definir um valor qualquer para as variáveis de altura e de raio. Agora vamos escrever a fórmula, conforme dita no enunciado e, em seguida, exibir na tela uma mensagem contendo o resultado do volume encontrado.